

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN: PAGE TABLE AND FILE SYSTEM

MÔN HỌC: Hệ Điều Hành

LỚP: 24C04

Thành viên

Trương Khánh Duy
24127160

Nguyễn Trọng Khang
24127411

Nguyễn Thị Hồng Trúc
24127575

Giáo viên hướng dẫn

Ths.Lê Đức Hoà

Ts.Vũ Thị Mỹ Hằng

Thành phố Hồ Chí Minh, 2026

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Cô Vũ Thị Mỹ Hằng và Thầy Lê Đức Hòa – Giảng viên bộ môn Hệ điều hành, Khoa Công nghệ thông tin, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM.

Trong suốt quá trình học tập và thực hiện đồ án, chúng em đã nhận được sự chỉ dẫn tận tình, những kiến thức chuyên môn quý báu cùng những điều kiện học tập tốt nhất từ Thầy và Cô. Đây chính là nền tảng quan trọng giúp nhóm tiếp cận được với các công nghệ thực tế và hoàn thành đồ án này.

Đồ án "Page Table and File System" là kết quả của quá trình nỗ lực nghiên cứu và vận dụng lý thuyết vào thực tiễn. Tuy nhiên, do hạn chế về kiến thức và kinh nghiệm, đồ án khó tránh khỏi những thiếu sót. Nhóm kính mong nhận được những ý kiến đóng góp, nhận xét từ Thầy và Cô để sản phẩm được hoàn thiện hơn, đồng thời giúp chúng em rút ra những bài học kinh nghiệm quý giá cho chặng đường sắp tới.

Chúng em xin chân thành cảm ơn!

Thành phố Hồ Chí Minh, 2026

MỤC LỤC

A. THÔNG TIN NHÓM.....	4
1. Github:.....	4
2. Thông tin thành viên:	4
3. Phân chia nhiệm vụ thực hiện và mức độ hoàn thành:.....	4
4. Phân chia điểm cho từng thành viên:	4
B. THỰC HIỆN	5
1.Task 1 : SPEED UP SYSTEM CALLS	5
1.1.Mục tiêu :.....	5
1.2.Các hàm nền tảng được sử dụng	5
1.3.Quá trình thực hiện :.....	5
1.4.Kết quả thực hiện :	6
2. PRINT PAGE TABLE (vmprint):	6
2.1. Tổng quan:.....	6
2.2. Cấu trúc dữ liệu:	7
2.3. Các bước triển khai:	7
2.4. Kiểm thử chương trình:	9

A. THÔNG TIN NHÓM

1. Github: <https://github.com/NguyenKhang15/Project2-HeDieuHanh>

2. Thông tin thành viên:

STT	MSSV	Họ và tên	Vai trò
1	24127575	Nguyễn Thị Hồng Trúc	Member
2	24127411	Nguyễn Trọng Khang	Leader
3	24127160	Trương Khánh Duy	Member

3. Phân chia nhiệm vụ thực hiện và mức độ hoàn thành:

	STT Họ và tên	Nhiệm vụ	Mức độ hoàn thành
1	Nguyễn Thị Hồng Trúc	Task 2 :PRINT PAGE TABLE	100%
2	Nguyễn Trọng Khang	-Task 1 : SPEED UP SYSTEM CALLS -Viết báo cáo	100%
3	Trương Khánh Duy	Task 3 : LARGE FILE	100%

4. Phân chia điểm cho từng thành viên:

STT	Họ và tên	Mức điểm
1	Nguyễn Thị Hồng Trúc	33.33%

2	Nguyễn Trọng Khang	33.33%
3	Trương Khánh Duy	33.33%

B. THỰC HIỆN

1.Task 1 : SPEED UP SYSTEM CALLS

1.1.Mục tiêu :

- Mục tiêu cốt lõi là thực hiện Tối ưu hóa system call `getpid()` bằng cách **loại bỏ việc chuyển đổi giữa user mode và kernel mode** (context switch)

1.2.Các hàm nền tảng được sử dụng

- `kalloc()`: Cấp phát một trang nhớ vật lý (4096 bytes) từ danh sách trang tự do của hệ thống.
- `mappages()`: Thiết lập ánh xạ từ địa chỉ ảo (Virtual Address) sang địa chỉ vật lý (Physical Address) trong bảng phân trang.
- `uvmunmap()`: Hủy bỏ ánh xạ giữa địa chỉ ảo và vật lý khi tiến trình không còn nhu cầu sử dụng.
- `kfree()`: Giải phóng trang nhớ vật lý trả lại cho hệ thống để tránh lỗi rò rỉ bộ nhớ (Memory Leak).

1.3.Quá trình thực hiện :

- Định nghĩa cấu trúc : trong `kernel/proc.h`, cấu trúc `struct proc` được bổ sung con trỏ `struct usyscall *usyscall`. Việc này giúp mỗi tiến trình có một tham chiếu riêng đến trang nhớ dùng chung ngay trong PCB
- Cấp phát tài nguyên :
 - Gọi `kalloc()` để lấy 1 trang vật lý.
 - Gán giá trị PID của tiến trình hiện tại vào trường `pid` của cấu trúc này: `p->usyscall->pid = p->pid`.
- Thiết lập ánh xạ : Kernel gọi `mappages()` để liên kết địa chỉ ảo `USYSCALL` với trang vật lý vừa cấp phát.
 - Permissions: Được set là `PTE_R | PTE_U` (Read-only từ phía User). Điều này đảm bảo tính toàn vẹn: User chỉ được

đọc PID, không được phép chỉnh sửa trái phép dữ liệu của Kernel.

- Dọn dẹp tài nguyên : Hàm kfree() được gọi để thu hồi trang nhớ vật lý đã cấp cho usyscall.
- Giải phóng bảng phân trang : Trước khi bảng phân trang của tiến trình bị hủy, hàm uvmunmap() được gọi tại địa chỉ USYSCALL. Thao tác này đảm bảo bộ nhớ ảo được dọn dẹp sạch sẽ trước khi tiến trình hoàn toàn biến mất khỏi hệ thống.

1.4. Kết quả thực hiện :

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x1000 pte 0x21FC7017 pa 0x87F1C000 perm 0x17
va 0x2000 pte 0x21FC6C07 pa 0x87F1B000 perm 0x7
va 0x3000 pte 0x21FC68D7 pa 0x87F1A000 perm 0xD7
va 0x4000 pte 0x0 pa 0x0 perm 0x0
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFD000 pte 0x21FD4C13 pa 0x87F53000 perm 0x13
va 0xFFFFE000 pte 0x21FD00C7 pa 0x87F40000 perm 0xC7
va 0xFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
print_kpgtbl: OK
superpg_test starting
pgtbltest: superpg_test failed: pte different, pid=3
```

- print_pgtbl: OK (Pass bài in Page Table)
- ugetpid_test: OK Xác nhận tiến trình người dùng có thể đọc PID trực tiếp mà không cần sự can thiệp của Kernel qua mỗi lần gọi.
- print_kpgtbl: OK Xác nhận việc chèn trang USYSCALL vào bảng phân trang là chính xác và không gây xung đột với các vùng nhớ khác như TRAPFRAME hay TRAMPOLINE

2. PRINT PAGE TABLE (vmprint):

2.1. Tổng quan:

Triển khai một hàm vmprint() trong kernel có khả năng in ra cấu trúc phân cấp của bảng trang cho một tiến trình. Hàm này giúp debug và hiểu rõ cách quản lý bộ nhớ ảo trong xv6.

Mục tiêu là viết hàm vmprint() có thể duyệt qua bảng trang 3 cấp của xv6 (dựa trên kiến trúc RISC-V Sv39) và in ra tất cả các mục (Page Table Entry - PTE) hợp lệ, bao gồm cả các mục trỏ đến bảng trang con ở cấp độ sâu hơn, với định dạng thụt đầu dòng bằng dấu ".." để thể hiện cấp độ.

2.2. Cấu trúc dữ liệu:

Page table trong xv6 được tổ chức dưới dạng cây 3 cấp theo chuẩn RISC-V Sv39:

Cấp độ	Tên gọi	Vai trò
Level 2	Root page table	Cấp cao nhất, trỏ đến các page table cấp 1
Level 1	Middle page table	Cấp trung gian, trỏ đến các page table cấp 0
Level 0	Leaf page table	Cấp thấp nhất, trỏ đến các trang dữ liệu thật

Mỗi page table có 512 entry, mỗi entry là một PTE 64-bit

Các macro và hằng số sử dụng được định nghĩa trong kernel/riscv.h

2.3. Các bước triển khai:

2.3.1. Khai báo hàm vmprint trong kernel/defs.h:



```
Open ▾ [+]
```

```
defs.h  
~/Documents/Project2-HeDieuHanh/kernel  
def.h
```

```
int copyout(pagetable_t, uint64, char *, uint64);  
int copyin(pagetable_t, char *, uint64, uint64);  
int copyinstr(pagetable_t, char *, uint64, uint64);  
#if defined(LAB_PGTBL) || defined(SOL_MMAP)  
void vmprint(pagetable_t);  
#endif  
#ifdef LAB_PGTBL  
pte_t* pgpte(pagetable_t, uint64);  
#endif
```

2.3.2. Cài đặt hàm vmprint() trong kernel/vm.c:

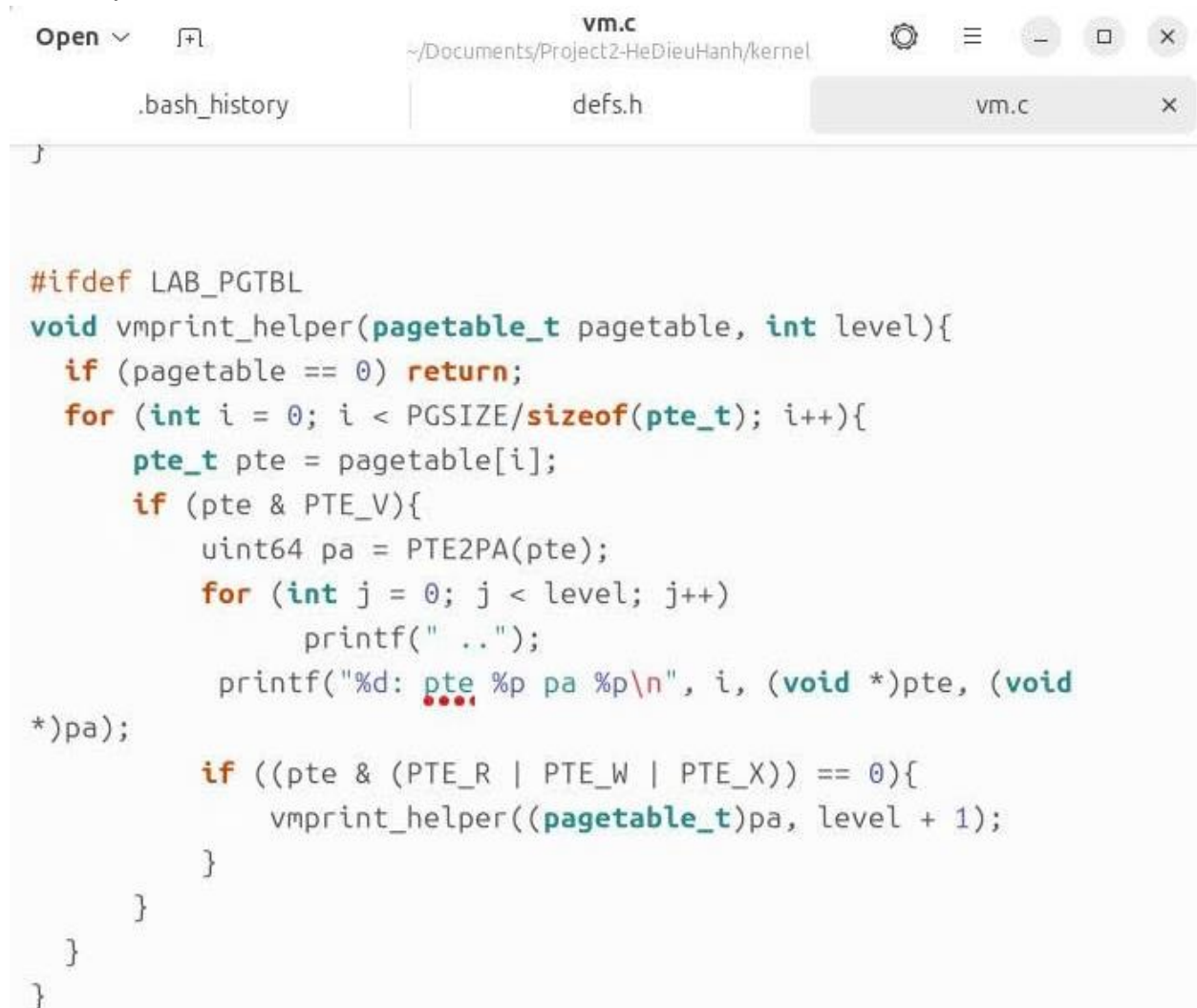
Logic thực hiện:

1. Duyệt qua 512 entries trong page table
2. Với mỗi entry valid (có bit V):
 - In indent (số cấp `..` = level)
 - In số thứ tự entry, giá trị PTE, và địa chỉ vật lý
 - Nếu không có bit R/W/X → đây là pointer đến page table con → gọi đệ quy xuống level tiếp theo

- Nếu có bit R/W/X → đây là leaf page (page thật) → không đệ quy nữa

Chi tiết mã nguồn:

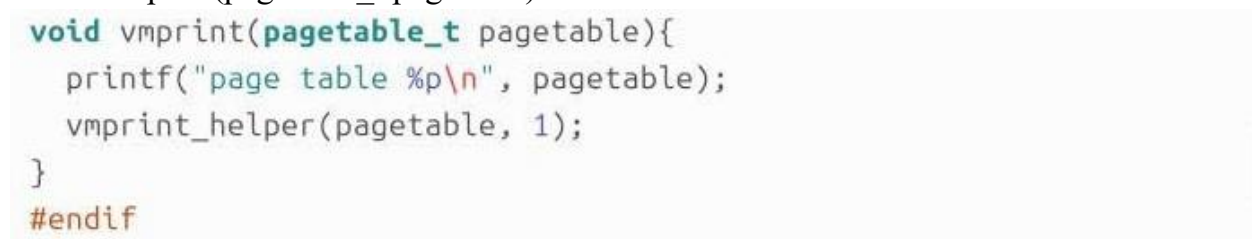
*Hàm `vmprint_helper(pagetable_t pagetable, int level)`: Hàm đệ quy để duyệt và in các entry



```
vm.c
~/Documents/Project2-HeDieuHanh/kernel
.bash_history  defs.h  vm.c

#ifdef LAB_PGTBL
void vmprint_helper(pagetable_t pagetable, int level){
    if (pagetable == 0) return;
    for (int i = 0; i < PGSIZE/sizeof(pte_t); i++){
        pte_t pte = pagetable[i];
        if (pte & PTE_V){
            uint64 pa = PTE2PA(pte);
            for (int j = 0; j < level; j++){
                printf(" ..");
                printf("%d: pte %p pa %p\n", i, (void *)pte, (void
*)pa);
            if ((pte & (PTE_R | PTE_W | PTE_X)) == 0){
                vmprint_helper((pagetable_t)pa, level + 1);
            }
        }
    }
}
#endif
```

*Hàm `vmprint(pagetable_t pagetable)`: Hàm chính



```
void vmprint(pagetable_t pagetable){
    printf("page table %p\n", pagetable);
    vmprint_helper(pagetable, 1);
}

#endif
```

2.3.3. Gọi `vmprint()` trong `kernel/exec.c`:

Thêm vào cuối hàm `exec()`, trước `return argc`: nếu `p->pid == 1` thì gọi `vmprint(p->pagetable)`



```
Open ▾ [🔍] exec.c
~/Documents/Project2-HeDieuHanh/kernel

.bash_history | defs.h | vm.c | exec.c x

p->SZ = SZ;
p->trapframe->epc = elf.entry; // initial program counter = main
p->trapframe->sp = sp; // initial stack pointer
proc_freepagetable(oldpagetable, oldsz);

if(p->pid == 1){
    vmprint(p->pagetable);
}

return argc; // this ends up in a0, the first argument to
main(argc, argv)

bad:
if(pagetable)
    proc_freepagetable(pagetable, sz);
if(ip){
    iunlockput(ip);
end op();
```

2.4. Kiểm thử chương trình:

Để kiểm chứng hàm `vmprint()`, ta không cần viết chương trình người dùng riêng. Hàm được gọi tự động từ `exec.c` khi process init (`pid = 1`) được tạo. Chỉ cần chạy `make qemu`, hàm sẽ tự động thực thi và in ra cấu trúc page table.

Các bước kiểm thử:

1. Biên dịch lại kernel:

- `make clean`
- `make qemu`

2. Output:

```
ubuntu@ubuntu2204: ~/Documents/Project2-HeDieuHanh
54 total 2000
ballocc: first 823 blocks have been allocated
ballocc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp
3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,f
ormat=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
page table 0x0000000087f4e000
..0: pte 0x0000000021fd2801 pa 0x0000000087f4a000
.. ..0: pte 0x0000000021fd2401 pa 0x0000000087f49000
.. .. ..0: pte 0x0000000021fd2c1b pa 0x0000000087f4b000
.. .. ..1: pte 0x0000000021fd2017 pa 0x0000000087f48000
.. .. ..2: pte 0x0000000021fd1c07 pa 0x0000000087f47000
.. .. ..3: pte 0x0000000021fd1817 pa 0x0000000087f46000
..255: pte 0x0000000021fd3401 pa 0x0000000087f4d000
.. ..511: pte 0x0000000021fd3001 pa 0x0000000087f4c000
.. .. ..510: pte 0x0000000021fd5807 pa 0x0000000087f56000
.. .. ..511: pte 0x000000002000180b pa 0x0000000080006000
init: starting sh
$
```

3: Large Files

3.1 Mục tiêu (Objective)

- Nâng cấp hệ thống tệp của xv6 để phá vỡ giới hạn 268 blocks ban đầu.
- Mục tiêu cụ thể: Hỗ trợ tệp tin có kích thước tối đa lên đến 65803 blocks bằng cách thêm cấu trúc doubly-indirect block.

3.2 Thiết kế và Giải pháp (Design & Solution)

- Cấu trúc Inode mới: Thay đổi từ 12 khối trực tiếp xuống còn 11 khối trực tiếp.
- Singly-indirect block: Giữ nguyên 1 khối (quản lý 256 data blocks).
- Doubly-indirect block: Thêm 1 khối tại `addrs[12]`. Khối này trỏ đến 256 khối gián tiếp đơn, mỗi khối đó lại trỏ đến 256 data blocks.
- Công thức tính: $11 \text{ (direct)} + 256 \text{ (singly)} + 256 * 256 \text{ (doubly)} = 65803 \text{ blocks}$.

3.3 Các bước thực hiện (Implementation)

- **kernel/fs.h & kernel/file.h:** Sửa NDIRECT thành 11 và cập nhật MAXFILE lên 65803.

- **kernel/fs.c (hàm bmap):** Lập trình logic để ánh xạ địa chỉ cho tầng gián tiếp kép (doubly-indirect).
- **kernel/fs.c (hàm itrunc):** Bổ sung vòng lặp 3 tầng để giải phóng các khối dữ liệu và khối trung gian trong vùng doubly-indirect.

3.4 Kết quả kiểm thử

- Kết quả chạy bigfile

```
wrote 65803 blocks
reading bigfile
.....
.....
.....
.....
.....
.....
.....
.....
.....
bigfile done; ok
```

- Kết quả chạy usertests

```
test outofinodes: ialloc: no inodes
OK
ALL TESTS PASSED
$ QEMU: Terminated
```